基于 MongoDB 的问卷系统实验文档（阶段一）

1. 实验目的

本实验旨在完成一个简化版在线问卷系统，核心目标如下：

1. 基于 MongoDB 完成问卷业务的数据建模。
2. 实现问卷系统核心后端逻辑，包括用户、问卷、题目、填写、统计。
3. 支持条件跳转与答案校验，保证填写数据有效性。
4. 形成可运行、可说明、可测试的课程项目交付材料。

2. 项目需求理解

根据课程要求，系统需要覆盖以下能力：

1. 用户账号：注册、登录、按账号隔离问卷数据。
2. 问卷管理：创建、查看列表、发布、关闭、设置截止时间、分享链接。
3. 题型支持：单选、多选、文本填空、数字填空（通过 text_input 与 number_input 实现）。
4. 校验逻辑：必答、多选数量约束、文本长度约束、数字范围与整数约束。
5. 跳转逻辑：根据单选、多选、数字条件跳转到目标题。
6. 填写提交：支持匿名/登录填写、支持多人填写、保存每次提交。
7. 统计分析：整卷统计与单题统计，覆盖选项计数、文本列表、数字平均值。

课程额外强调：

1. 所有数据必须存储在 MongoDB。
2. 第二阶段必须在第一阶段基础上迭代，不可推倒重做。
3. 需保留 AI 使用过程记录与测试记录。

3. 当前项目内容概览

3.1 技术栈

1. 后端：Node.js + Express 5 + Mongoose。
2. 数据库：MongoDB。
3. 认证：JWT。
4. 前端：public 目录下的原生 HTML/CSS/JS 页面。

3.2 模块结构（与代码一致）

1. server/app.js：服务入口、静态资源托管、路由挂载、全局错误处理。
2. server/models：User、Survey、Question、Response 四类核心数据模型。
3. server/routes：auth、surveys、questions、fill、stats 五大业务路由。
4. server/utils：validation（答案校验）、jumpLogic（跳转规则引擎）。
5. public/js：auth、dashboard、create、fill、stats 五个前端交互脚本。

4. 系统设计

4.1 架构说明

系统采用前后端分层：

1. 浏览器访问 public 静态页面。
2. 前端通过 Fetch 调用后端 REST API。
3. 后端通过 Mongoose 操作 MongoDB。
4. JWT 由前端持久化并随请求头传递，实现鉴权。

4.2 主要业务流程

1. 注册登录：用户注册后生成 Token，登录后进入我的问卷。
2. 问卷设计：创建草稿问卷，添加题目及规则，发布后生成可分发链接。
3. 问卷填写：访问分享链接，系统检查发布状态、截止时间和匿名策略。
4. 数据统计：创建者查看整卷与单题统计结果。

5. MongoDB 数据库设计

5.1 users 集合

核心字段：

1. username：唯一用户名。
2. password：bcrypt 哈希密码。
3. createdAt：注册时间。

设计要点：

1. 用户名唯一索引保证账号唯一性。
2. 使用 pre save 中间件自动加密密码。

5.2 surveys 集合

核心字段：

1. creatorId：创建者用户 ID。
2. title、description：问卷基本信息。
3. allowAnonymous：是否允许匿名。
4. status：draft、published、closed。
5. deadline：截止时间。
6. shareCode：分享码（唯一）。

设计要点：

1. creatorId 建索引以支持我的问卷快速查询。
2. shareCode 唯一索引用于分享链接访问。

5.3 questions 集合

核心字段：

1. surveyId：所属问卷。

2. order：题目顺序。
3. type：single_choice、multiple_choice、text_input、number_input。
4. title、required、options。
5. validation：各题型校验参数。
6. jumpRules：条件与目标题号。

设计要点：

1. questions 独立集合，避免单问卷文档过大。
2. 使用 surveyId + order 复合索引提升按序读题效率。
3. 跳转规则可配置，不写死在程序中。

5.4 responses 集合

核心字段：

1. surveyId：所属问卷。
2. respondentId：填写用户，可为空（匿名）。
3. answers：题目答案数组，支持混合类型。
4. submittedAt：提交时间。

设计要点：

1. 支持匿名、多人、多次提交。
2. 使用灵活的 value 类型存储不同题型答案。

5.5 为什么该设计适合 MongoDB

1. 题型与校验规则结构差异大，文档模型比关系型更自然。
2. 跳转规则是嵌套对象，MongoDB 表达更直接。
3. 答案值类型不固定，MongoDB 的灵活字段适配更好。
4. 问卷业务变更频繁，文档模型更利于迭代扩展。

6. 接口设计与实现情况

6.1 认证模块

1. POST /api/auth/register：注册。
2. POST /api/auth/login：登录。

6.2 问卷模块

1. POST /api/surveys：创建问卷。
2. GET /api/surveys：获取我的问卷。
3. GET /api/surveys/:id：问卷详情。
4. PUT /api/surveys/:id：更新问卷。
5. PUT /api/surveys/:id/publish：发布问卷。
6. PUT /api/surveys/:id/close：关闭问卷。
7. DELETE /api/surveys/:id：删除问卷并级联删除题目与回答。

6.3 题目模块

1. POST /api/surveys/:surveyId/questions：新增题目。
2. GET /api/surveys/:surveyId/questions：获取题目列表。
3. PUT /api/questions/:id：更新题目。
4. DELETE /api/questions/:id：删除题目。

6.4 填写模块

1. GET /api/survey/:shareCode：通过链接获取问卷。
2. POST /api/survey/:shareCode/submit：提交问卷。

6.5 统计模块

1. GET /api/surveys/:id/stats：整卷统计。
2. GET /api/surveys/:id/stats/:questionId：单题统计。

7. 关键逻辑说明

7.1 权限与状态控制

1. 问卷相关操作通过 JWT 鉴权。
2. 仅创建者可编辑、发布、关闭、删除、查看统计。
3. 仅草稿可改题，已发布问卷不可再改题。

7.2 提交校验

后端在提交时二次校验答案：

1. 必答题检查。
2. 单选选项合法性检查。
3. 多选数组合法性与 min、max、exact 约束检查。
4. 文本长度约束检查。
5. 数字类型、范围、整数约束检查。

7.3 跳转规则引擎

系统支持以下条件：

1. equals：值等于。
2. contains：多选包含。
3. gt、lt、gte、lte：数字比较。

执行逻辑：

1. 按题目顺序处理。
2. 对当前题按 jumpRules 顺序匹配，第一个命中规则生效。
3. 命中后跳到目标题号，否则进入默认下一题。

7.4 统计逻辑

1. 单选：按选项计数。
2. 多选：数组展开后按选项累计。

3. 文本：保留所有文本答案。
4. 数字：计算平均值、最小值、最大值。

8. 需求覆盖矩阵

需求点	当前实现状态	说明
注册登录	已实现	注册与登录 API 完整，密码哈希存储
仅查看自己的问卷	已实现	creatorId 与鉴权控制
创建问卷与发布关闭	已实现	草稿、发布、关闭状态流转完整
截止时间控制	已实现	填写时校验 deadline
单选/多选/填空题	已实现	text_input 与 number_input 覆盖填空场景
跳转逻辑可配置	已实现	jumpRules 存库并在提交时执行
提交时校验	已实现	后端 validation 二次校验
匿名填写	已实现	allowAnonymous 控制是否需要登录
统计整卷与单题	已实现	stats API 已支持两种统计
自动化测试	已实现	已完成 Jest + Supertest 端到端测试，12/12 通过

9. 实验环境与运行步骤

9.1 实验环境

1. 操作系统：Windows。
2. Node.js：建议 18 及以上。
3. MongoDB：建议 6.0 及以上。

9.2 运行步骤

1. 启动 MongoDB 服务。
2. 进入 server 目录并安装依赖。
3. 启动后端服务。
4. 浏览器访问 http://localhost:3000。
5. 注册账号，进入问卷管理与填写流程。

10. 测试设计

10.1 自动化测试现状

当前 server 中已具备可直接运行的自动化测试。 说明：

1. 测试文件：server/tests/survey-system.e2e.test.js。
2. 测试框架：Jest + Supertest。
3. 执行命令：在 server 目录运行 npm test。
4. 实际结果：1 个测试套件通过，12 个测试全部通过。

10.2 手工测试用例

以下用例覆盖课程要求的最低测试范围。

用例 1：创建问卷测试

1. 步骤：登录后创建问卷，填写标题与说明并保存。
2. 输入：标题“课程反馈”，匿名开关任意。
3. 预期：创建成功并跳转到编辑页面。
4. 结果：通过（已由自动化用例覆盖）。

用例 2：添加题目测试

1. 步骤：在草稿问卷中添加单选、多选、文本、数字题各 1 道。
2. 输入：多选设置最少 2、最多 3；数字设置 0 到 120、整数。
3. 预期：题目保存成功，列表按顺序显示。
4. 结果：通过（已由自动化用例覆盖）。

用例 3：跳转逻辑测试

1. 步骤：为第 1 题设置条件跳转，例如选 A 跳第 3 题。
2. 输入：填写时第 1 题选择 A。
3. 预期：后端按规则跳过非目标路径题目进行校验与记录。
4. 结果：通过（已由自动化用例覆盖）。

用例 4：校验测试

1. 步骤：提交不符合规则的答案。
2. 输入：必答题留空、多选超出限制、数字超范围。
3. 预期：返回对应错误提示并拒绝提交。
4. 结果：通过（已由自动化用例覆盖）。

用例 5：提交问卷测试

1. 步骤：按规则填写并提交。
2. 输入：所有必答题合法。
3. 预期：返回提交成功与 responseId，数据库新增 response 文档。
4. 结果：通过（已由自动化用例覆盖）。

用例 6：统计测试

1. 步骤：同一问卷提交多份回答后查看统计页。
2. 输入：不同用户选择不同选项并填写数字。
3. 预期：选项计数正确，数字平均值正确，文本答案可见。
4. 结果：通过（已由自动化用例覆盖）。

10.3 自动化测试执行摘要

1. 执行目录：server
2. 执行命令：npm test
3. 结果：

1. Test Suites: 1 passed, 1 total
2. Tests: 12 passed, 12 total
3. Time: 约 3.5 秒

11. 实验结果与分析

11.1 已完成能力

1. 后端核心业务链路完整，可支持问卷从创建到统计的全流程。
2. MongoDB 建模符合文档型场景，满足课程核心考核点。
3. 跳转逻辑与答案校验已服务端落地，能保障数据质量。

11.2 当前不足

1. 自动化测试已覆盖核心流程，但仍可继续增加异常边界与并发场景。
2. 前端填写页当前为整体展示，跳转体验可进一步可视化优化。

12. 遇到的问题与如何解决

12.1 遇到的问题

1. 问卷提交链路在存在特殊跳转配置时，可能出现路径循环风险。
2. 跳转条件比较在字符串与数字混用场景下，容易产生前后端行为不一致。
3. 删除题目后，其他题目的跳转规则可能残留失效目标，造成后续跳转异常。
4. 初始阶段自动化测试覆盖不足，难以稳定验证核心流程正确性。
5. 用户侧缺少账户注册时间可视化入口，不利于满足完整功能体验。

12.2 如何解决

1. 在提交流程加入防循环策略（访问标记与最大步数限制），避免接口卡死。
2. 统一跳转逻辑中的条件比较方式，并增强数字比较鲁棒性，降低类型误判。
3. 在删除题目时同步清理指向该题的跳转规则，保持问卷结构一致性。
4. 补充 Jest + Supertest 端到端测试，覆盖创建、题目、跳转、校验、提交、统计六类关键场景。
5. 在 UI 中增加“点击右上角用户名查看账户信息”入口，并展示用户注册时间。

13. 结论

本项目已实现问卷系统第一阶段的主要功能与核心后端逻辑，整体设计与 MongoDB 技术路线匹配，能够支撑后续需求变更迭代。

==仓库地址==

本文档仅展示部分的实验结果与内容，详细代码、测试样例细节、启动说明、ai使用报告、核心文档见代码仓库。

- `public/`：前端静态页面与样式、脚本文件。
- `server/`：后端服务代码（Express + MongoDB），包含路由、模型、中间件与工具函数。
- `docs/`：测试与说明文档。

https://github.com/Kevin23-design/DataManagementSystem-26sp